TIVIDY Final Report — COS 214

TIVIDY is a generic workflow management system. A project is a tree of work that moves through a lifecycle, is done by employees, and reports its progress up the tree. The engine is generic, and the two industries in this report are two uses of it.

1. Team information

GitHub repository: https://github.com/Brett-Pitts/COS214Project

Team name: Vibe Engineers. Team leader: Jamie King.

Member	Student number
Rudolph Botha	u25387023
Francois Venter	u25555202
Louwrens Johansen	u25607414
Brett Pitts	u24682251
Jamie King	u24916031

main is the release branch and dev is the integration branch. Each task is done on its own branch (for example Employee-Builder, Employee-Composite-Decorator, Work-Iterator, Uml) and merged through a pull request that another member reviews once the task is finished. A branch that depends on another branch waits until that one is merged and then pulls the update. Every design decision is recorded in the Design Decisions and Revision History document. Closed pull requests: 48.

2. Workflow research, assumptions and boundaries

• What is a workflow and what is a workflow management system?

A workflow is a system for managing repetitive processes and tasks which occur in a particular order [1]. A workflow management system is a unified software platform that automates the sequence of tasks within a process – routing work to the right people, at the right time, based on rules your team defines [2].

• What is the difference between a workflow definition and a running workflow instance?

The workflow definition holds the rules and structure if the workflow. It is the template of with the workflow instance is built. The workflow definition is static [3]. A running workflow instance holds a runtime state: Current phase, assigned members, case data, timestamps, audit trial, etc.

• How are activities or work items represented?

Activities are represented as atomic(leaf) or a composite structure. This allows nesting of activities and subprocesses to easily be stored.

• How are participants, roles and responsibilities represented?

Participants have roles/capabilities that represent what kind of work they can do. They can then be assigned responsibilities based on their capabilities.

• How are work items assigned and routed?

Work items can be assigned based on various distribution rules e.g. round robin, lowest workload, etc. Work is routed with logic and notifications to allow members on the next part to start when the previous part is done.

• How are approvals, rejection and escalation handled?

There are different roles and levels of seniority. Escalations are then sent to someone who has a higher position. Approvals and rejections are handled with the implemented logic routing.

• How do workflow systems interact with external systems?

Workflow systems interact with external systems through interfaces, Api calls, connectors, etc to request and receive information.

• How are dependencies, events and failures handled?

Failures are handled in different ways. The process can be tried again, rolled back or with specific exception handling. Events can start, stop or change a process.

• How is workflow history or audit information recorded?

History is recorded in a log of changes. By combining the log with snapshots of the system. Changes can be rolled back and recovery becomes easy.

• How are running workflows monitored and reported on?

Running workflows are monitored through a dashboard. Changes to the workflow are reported to the dashboard. Using external systems such as gmail, changes can be reported the relevant people through email.

• What makes a workflow engine reusable across different processes?

By only implementing generic patterns, workflow engines can be used by different processes. More detail and the specifics of the process are made to be part of the workflow definition.

• Which decisions are normally configurable rather than hard coded into the workflow engine?

The process structure, role assignment and assignment strategies, process data, deadlines for the phases and the external programs used are examples of things that must be configurable.

•How are changes to workflow definitions handled while work is already in progress?

There are various strategies including: The current process stays on the old version (version pinning) [4], and new processes use the new definition when created. Migrating to the updated definition at safe points (e.g. At snapshots).

References:

[1] IBM. (2021, October 14). Workflow. Ibm.Com. https://www.ibm.com/think/topics/workflow

[2] admin. (2018, May 25). What is Workflow Management System and how it works? Cflow. https://www.cflowapps.com/workflow-management-system/

[3] IBM OpenPages. (2026, March 27). Ibm.Com. https://www.ibm.com/docs/en/openpages/9.2.x?topic=fundamentals-workflow-definitions workflow-instances

[4] pritesh. (2025, May 22). Version pinning in DevSecOps: A comprehensive tutorial. DevSecOps School. https://devsecopsschool.com/blog/version-pinning-in-devsecops-a-comprehensive-tutorial/

Assumptions:

- One running instance of TIVIDY holds one company and one workflow, and the workflow can contain any number of projects.
- Each work unit needs exactly one profession.
- Employees are added to a team in the order they should be asked to do work.
- Outside parties, such as the QA company and the municipal inspector, are modelled as ordinary teams that do work units.

Boundaries:

- No scheduling against calendar dates, no cost tracking and no payroll.
- No logins or access control.
- Runs as a single process on one thread.

3. Industry 1: game studio

A large video game company (for example Ubisoft) builds and launches one live-service game (for example Fortnite). The whole game is one project and each phase is a sub-project.

Participants: managers, marketing, HR, coders, artists, QA testers and IT staff, each in their own team. Every team has an intern, a junior and a senior.

Process:

1. Preparation: market research, then idea generation, then the budget, then hiring, each waiting for the one before it.
2. Creation: coding (player movement, world creation, story) and art (concept art, then final art) run in parallel. Starts after Preparation.
3. Marketing: trailer, gameplay showcase, interviews and ads. Starts after Creation.
4. QA testing: build testing by the QA team. Starts after Creation.
5. Servers: three servers, each acquired (bought, rented or reallocated) and then set up. Starts after Creation.
6. Launch: launching the game, once Marketing, QA testing and Servers are done.

Decisions: whether the idea is approved, whether money is left after Creation, whether the deadline is met and whether QA finds bugs. The engine does not branch on these. A user acts on them through the interface by cancelling, failing or forcing a work unit.

Failure paths: if the idea is rejected, the money runs out or the deadline is missed, the user cancels the affected work units. If nobody in the assigned team can move a unit on, it goes back to Ready to be retried, and it becomes Failed once its retries run out. A user can also force a unit to fail.

Input: the scenario is built by ApplicationInterface::createScenarioA() (./TIVIDY ScenarioA). It can then be saved to and loaded from text files.

Result: Launch Game is Completed and the Fortnite project is Completed. The task page shows the state of every work unit.

See activity diagram 3 in section 7.3.

4. Industry 2: house construction

A building contractor builds one house on the same engine, using different data.

Participants: architects, engineers, bricklayers, plumbers, electricians and municipal inspectors, each in their own team with an intern, a junior and a senior, plus a Managers team with one senior site manager.

Process:

1. Design: site survey (engineer), then plans (architect).
2. Structure: foundations, then framing (bricklayer). Needs the plans.
3. Services: plumbing and electrical in parallel. Needs the structure.
4. Municipal inspection (inspectors). Needs the services.
5. Handover (architect). Needs the inspection.

Decisions: whether the inspection passes, whether the client continues and whether money is left after Structure. As in the game studio, a user acts on these through the interface.

Failure paths: if the inspection fails or the client withdraws, the user fails or cancels the affected work units. If nobody qualified is available, the unit goes back to Ready to be retried, and it becomes Failed once its retries run out.

Input: the scenario is built by ApplicationInterface::createScenarioB() (./TIVIDY ScenarioB), using the same engine classes as the game studio. It is saved and loaded through the same interface.

Result: the Handover work unit is Completed and the House project is Completed.

See activity diagram 4 in section 7.4.

5. Generic engine and industry data

All classes in src/ are generic. Project (composite) and WorkUnit (leaf) share WorkNode, and the tree itself is the workflow, so there is no separate workflow template. The two industries only differ in their data, and no source file changes between them.

Engine part	Classes
Work tree and states	WorkNode, Project, WorkUnit, WorkState and the seven states
Finding work	Aggregate, Iterator, WorkIterator
Company structure	CompanyNode, Department, Team, TeamIterator
Employees	EmployeeComponent, Employee, Manager, EmployeeDetail, EmployeeProfession, EmployeeCompetency, CompetencyLevel
Creating employees	EmployeeBuilder, ConcreteEmployeeBuilder, EmployeeDirector
Routing and messages	Mediator, WorkMediator, ConcreteWorkMediator
Saving and loading	Memento, MementoType, FileSystem
Entry point	ApplicationInterface
Interface (Qt)	MainWindow, MainMenuPage, CompanyPage, ProjectPage, TaskPage, LoadSavePage

Industry data	Game studio	Construction
Professions	Coder, Artist, Marketing, HR, QA, IT, Manager	Architect, Engineer, Bricklayer, Plumber, Electrician, Manager, Inspector

Built by	createScenarioA() (./TIVIDY ScenarioA)	createScenarioB() (./TIVIDY ScenarioB)
Saved as	company and projects text files	company and projects text files
Outside party	QA team	Inspectors team

Professions are registered at runtime and the competency levels are the fixed levels Intern, Junior and Senior, so a new industry does not need a new class.

6. Functional requirements and traceability

The test column gives the tag of the test in src/tests.cpp.

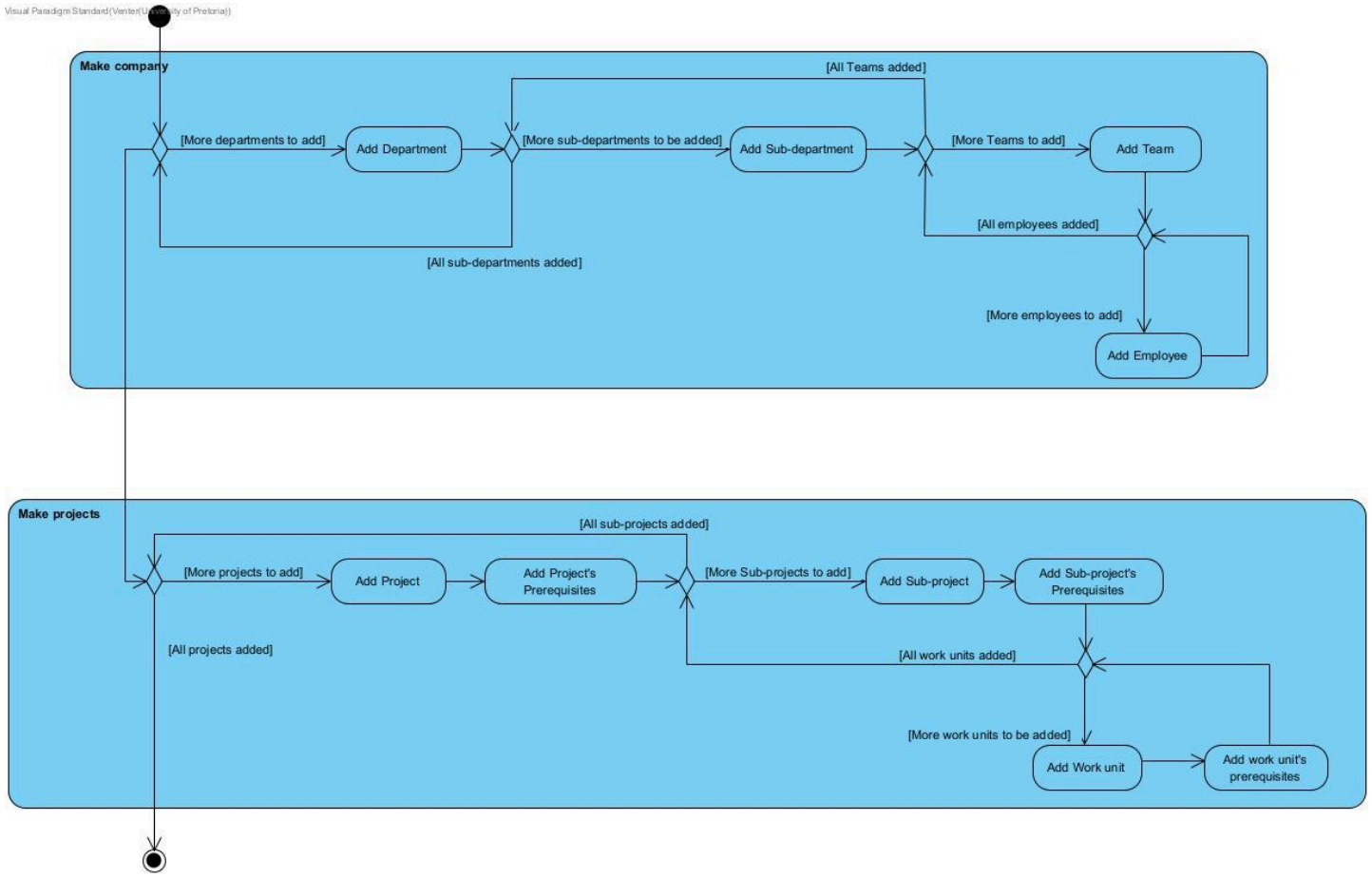
#	Requirement	Classes	Pattern	Test
FR1	Build a project as a tree of sub-projects and work units	WorkNode, Project, WorkUnit	Composite	[workflow], [worknode]
FR2	Build the company from departments and teams	CompanyNode, Department, Team	Composite	[company], [team]
FR3	Give employees professions and a competency level at runtime	EmployeeComponent, EmployeeDetail, EmployeeProfession, EmployeeCompetency	Decorator	[employee]
FR4	Create an employee from a name, profession and level	EmployeeBuilder, ConcreteEmployeeBuilder, EmployeeDirector	Builder	[employee], [company]
FR5	Find the next work unit that is ready	Aggregate, Iterator, WorkIterator	Iterator	[iterator]
FR6	Find a team for a profession	TeamIterator	Iterator	[iterator]
FR7	Pass a work unit along the team to the first employee who can do it	EmployeeComponent, Team	Chain of Responsibility	[chain], [team]
FR8	Do work in the order prepare, perform, finish	EmployeeComponent	Template Method	[employee]
FR9	Track each work unit through its states	WorkState and subclasses, WorkNode	State	[state], [worknode]

FR10	Wait for prerequisites and complete a project when its children are done	WorkNode, WorkMediator, ConcreteWorkMediator	Mediator	[mediator], [state]
FR11	Tell teammates when someone has worked on a unit	Team, Mediator, EmployeeComponent	Mediator	[team]
FR12	Retry work nobody can do, then fail it	Team, Failed, WorkNode	State, Chain of Responsibility	[team], [step]
FR13	Cancel a work unit	ApplicationInterface, Cancelled	State	[workflow]
FR14	Save and load the company and projects	Memento, FileSystem, ApplicationInterface	Memento	[saveload], [memento], [filesystem]
FR15	One entry point for the interface	ApplicationInterface	Facade	[workflow], [company], [saveload]
FR16	Use both industries from the interface	MainWindow, MainMenuPage, CompanyPage, ProjectPage, TaskPage, LoadSavePage	Facade	Demo

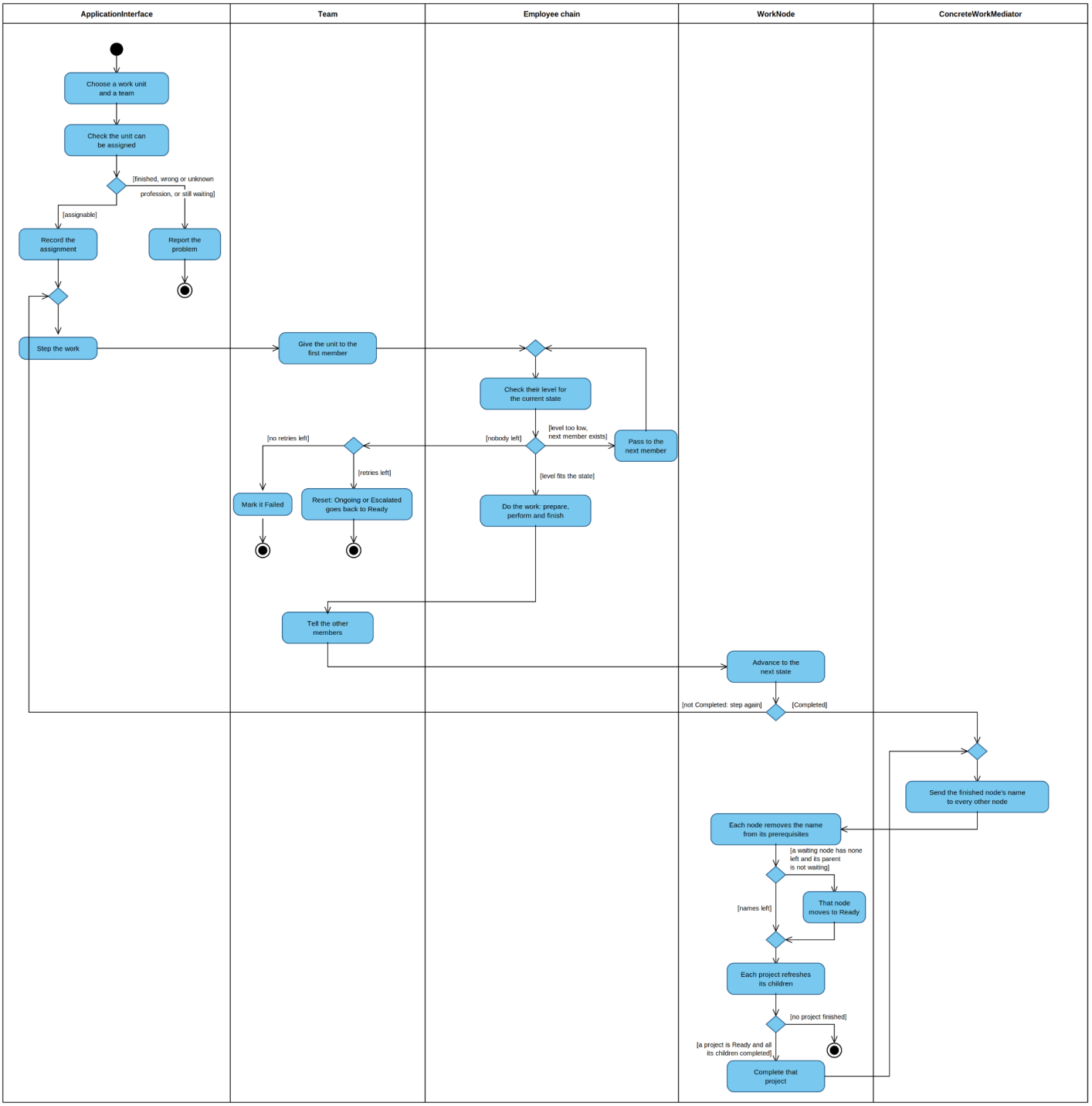
7. UML diagrams

The Visual Paradigm projects are diagrams/UML Diagrams/Class Diagram/UML.vpp and diagrams/UML Diagrams/Activity Diagrams/Activity Diagrams.vpp, with the exports next to them. Each pattern has its own colour on the class diagram.

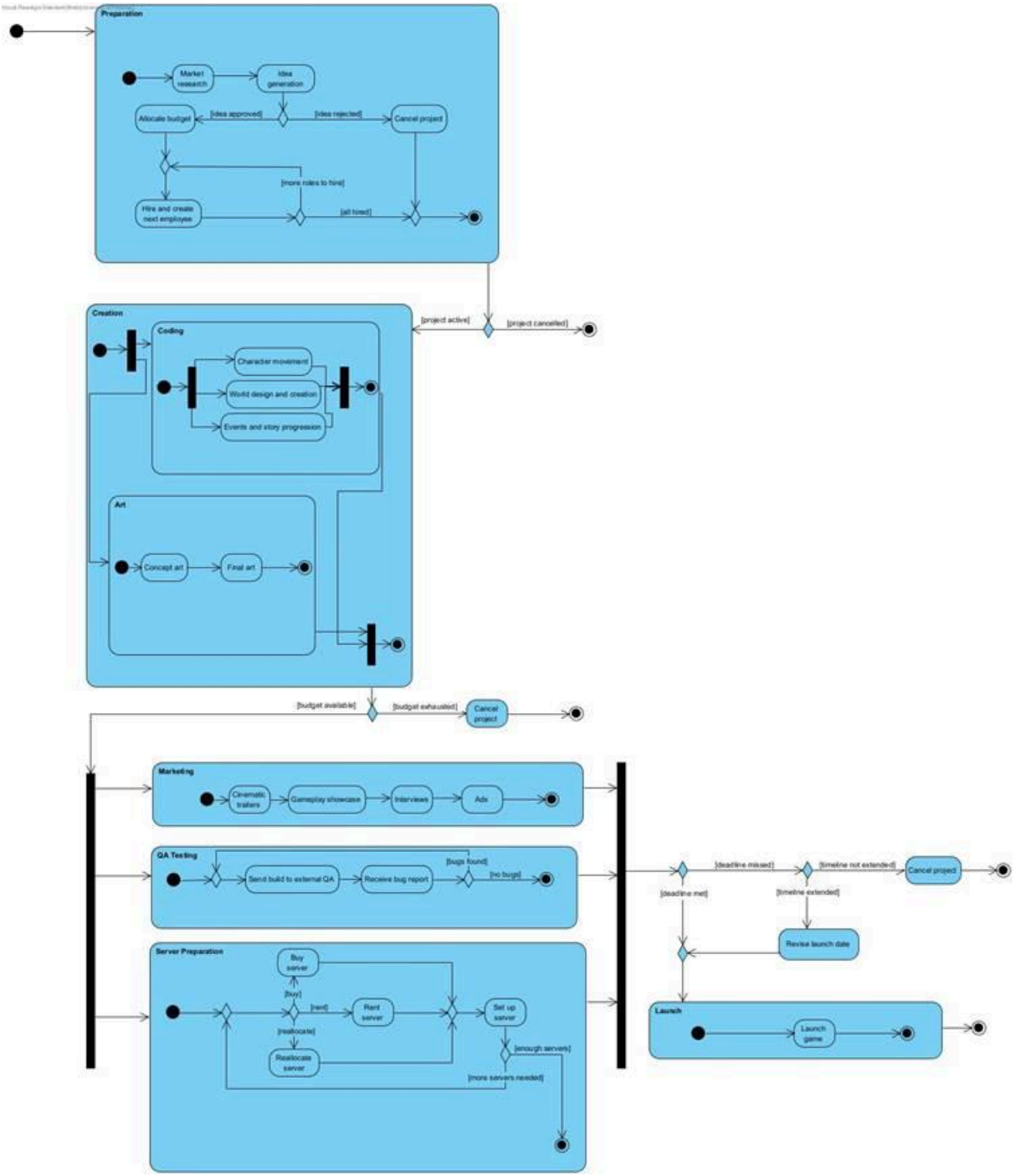
7.1 Activity diagram 1: Creation of Company and Projects



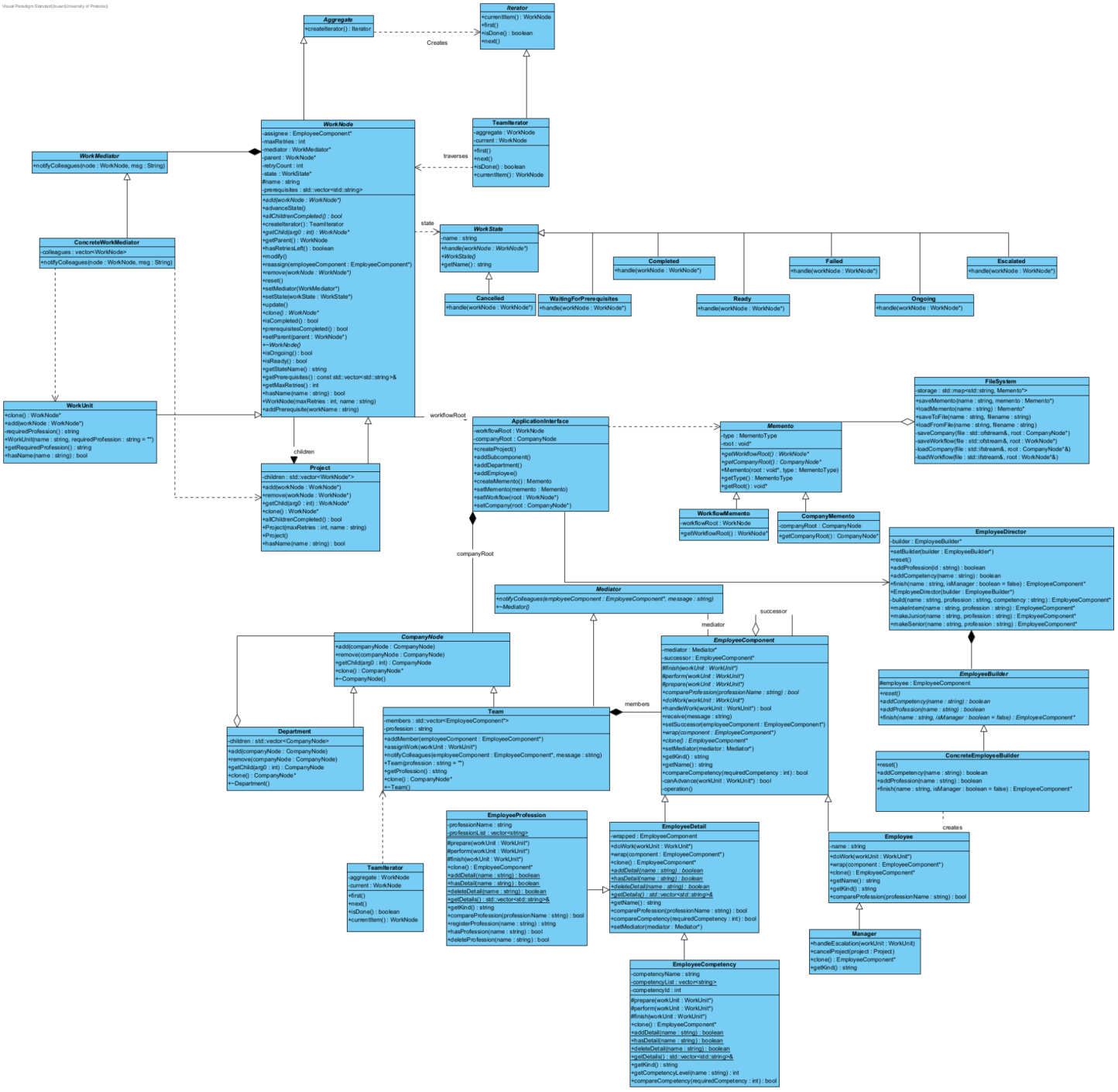
7.2 Activity diagram 2: assigning and completing work



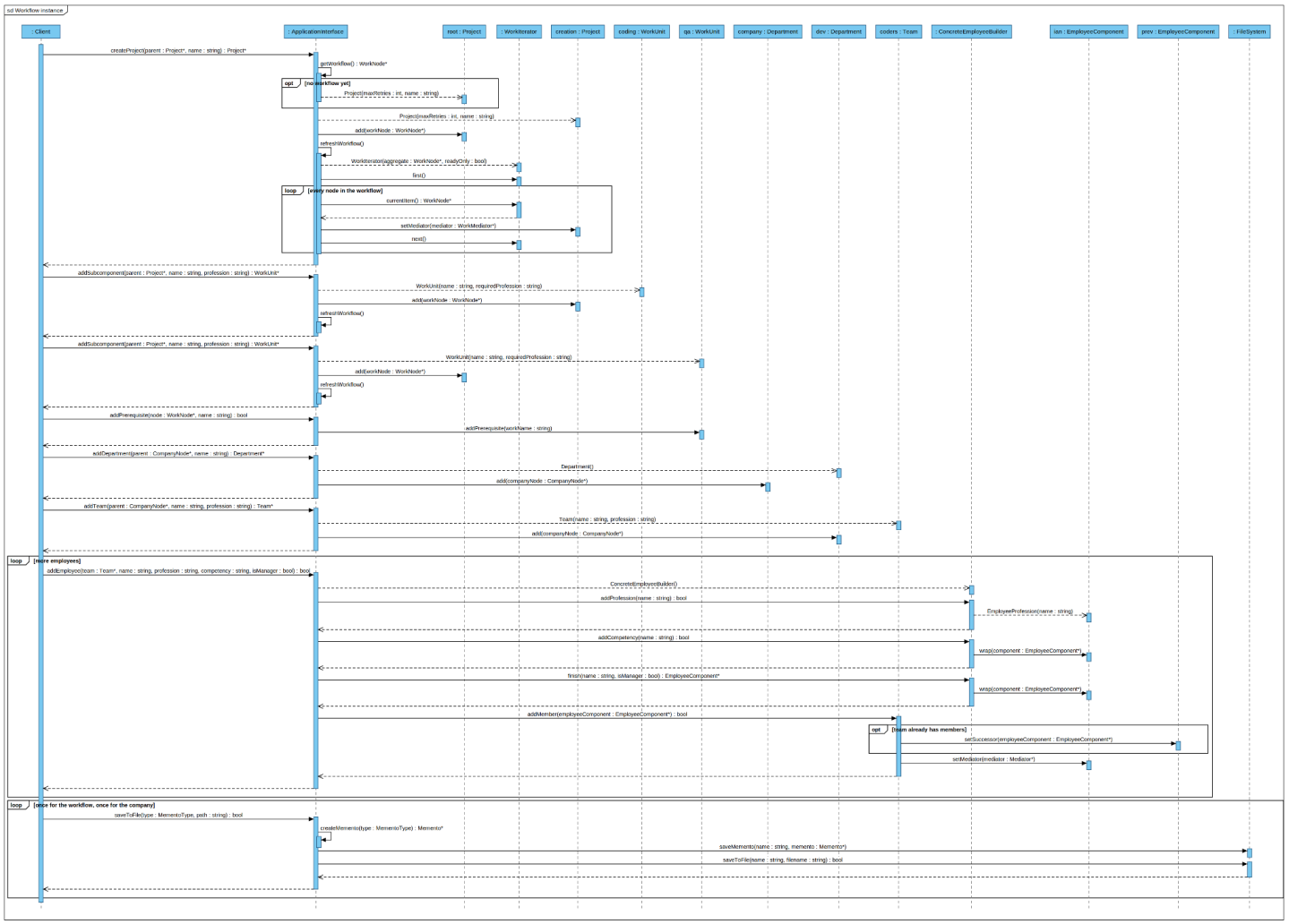
7.3 Activity diagram 3: game studio



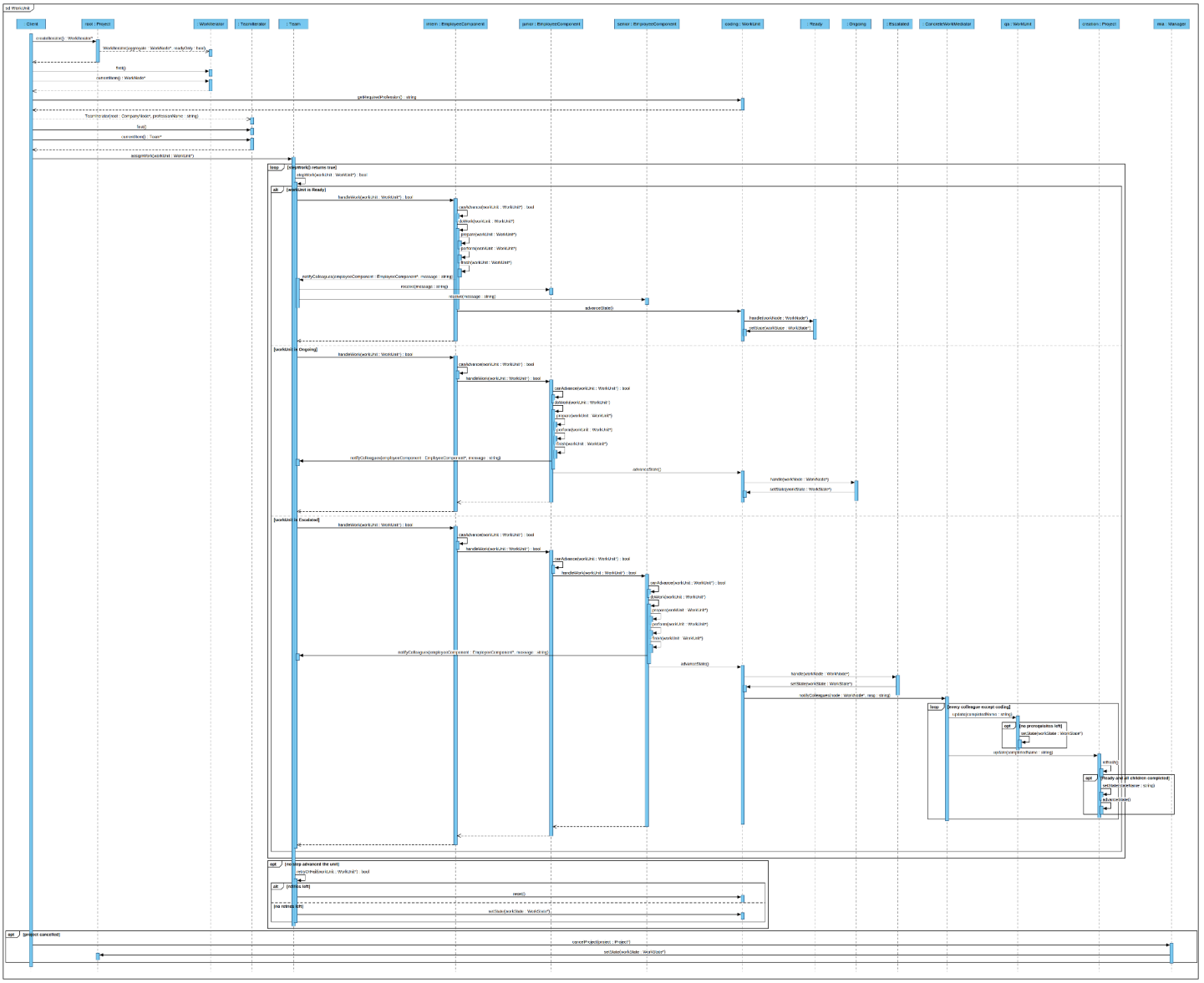
7.5 Class diagram



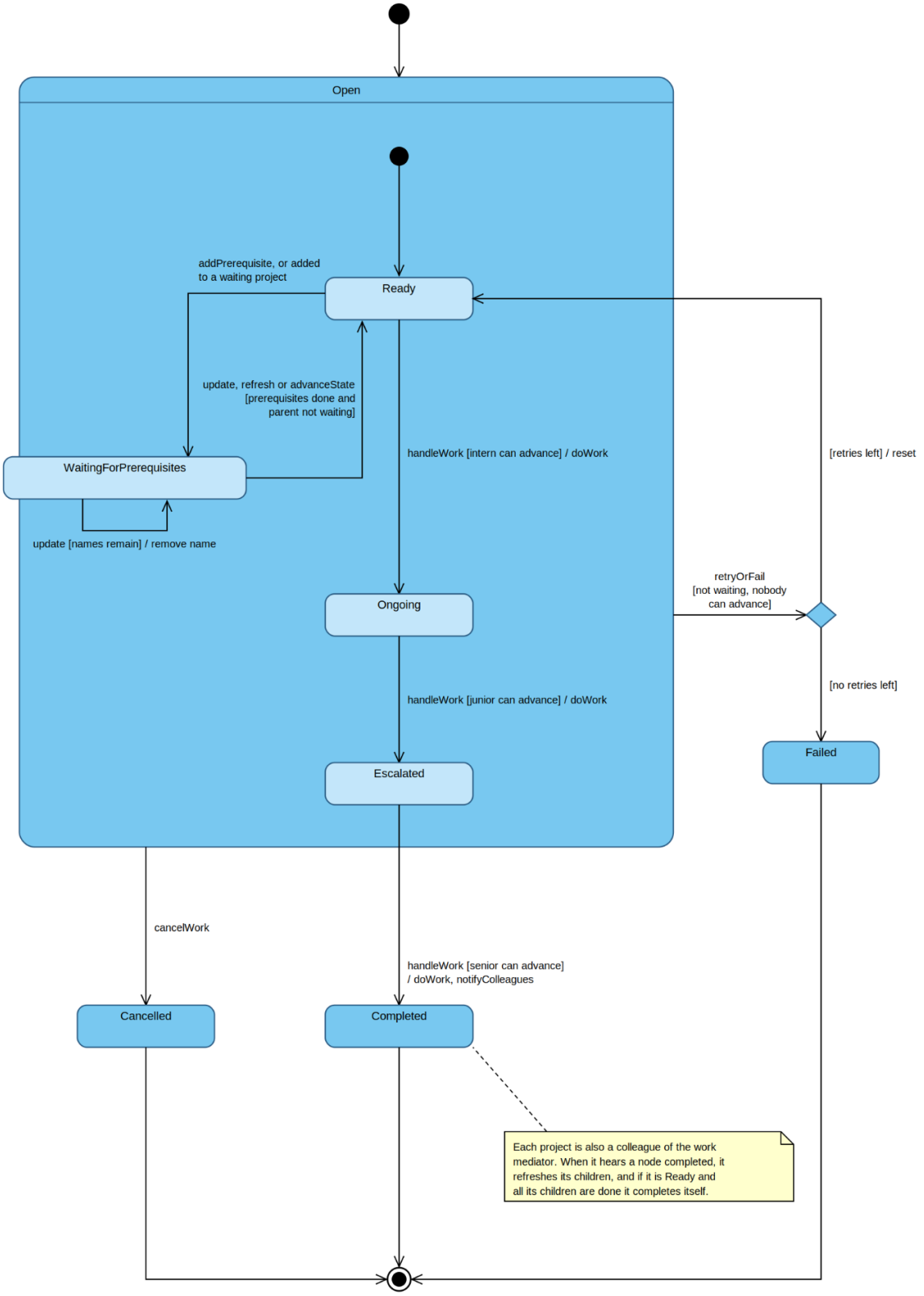
7.6 Sequence diagram 1: setting up a workflow instance



7.7 Sequence diagram 2: assigning and working a work unit



7.8 State diagram: work unit lifecycle



8. Design patterns

The system uses eleven patterns: two creational, three structural and six behavioural. Five are from the required list: Iterator, Mediator, Chain of Responsibility, Builder and Facade.

Pattern	Type	Why it is used	Participants

Composite (work tree)	Structural	A work unit and a whole project are treated the same, and completion moves up the tree	Component: WorkNode, Composite: Project, Leaf: WorkUnit
Composite (company)	Structural	Departments hold departments and teams, teams hold employees	Component: CompanyNode, Composite: Department, Leaf: Team
Decorator	Structural	Professions and levels are entered at runtime in any combination	Component: EmployeeComponent, ConcreteComponent: Employee, Manager, Decorator: EmployeeDetail, ConcreteDecorator: EmployeeProfession, EmployeeCompetency
Facade	Structural	One entry point for the interface	Facade: ApplicationInterface, Subsystem: FileSystem, ConcreteWorkMediator, EmployeeDirector, EmployeeBuilder, Memento
Builder	Creational	An employee is put together from a name, profession, level and manager flag	Builder: EmployeeBuilder, ConcreteBuilder: ConcreteEmployeeBuilder, Director: EmployeeDirector, Product: EmployeeComponent
Prototype	Creational	Employees and work trees are copied for saving	Prototype: EmployeeComponent, WorkNode, ConcretePrototype: Employee, Manager, the decorators, Project, WorkUnit
Memento	Behavioural	Save and load the company and projects without exposing their insides	Originator: ApplicationInterface, Memento: Memento, Caretaker: FileSystem
Iterator	Behavioural	Walk the work tree without exposing its children, stopping either on every node or only on Ready work units	Aggregate: Aggregate, ConcreteAggregate: WorkNode, Iterator: Iterator, ConcreteIterator: WorkIterator
Chain of Responsibility	Behavioural	A unit is passed along the team until someone with the right level takes it	Handler: EmployeeComponent, ConcreteHandler: each team member, linked in order by Team::addMember

Template Method	Behavioural	All work is done in the order prepare, perform, finish	AbstractClass: EmployeeComponent, ConcreteClass: EmployeeProfession, EmployeeCompetency, Employee
State	Behavioural	What happens when a unit moves on depends on its current state	Context: WorkNode, State: WorkState, ConcreteState: WaitingForPrerequisites, Ready, Ongoing, Escalated, Completed, Failed, Cancelled
Mediator	Behavioural	Team members are told about work done, and prerequisites release the units that wait on them	Mediator: Mediator, WorkMediator, ConcreteMediator: Team, ConcreteWorkMediator, Colleague: EmployeeComponent, WorkNode

Decorator instead of inheritance. Professions and levels are entered by the user and can be combined in any way. Inheritance would need a class for every combination. With decorators they are stacked at runtime, and the rest of the system only sees EmployeeComponent.

Iterator instead of exposing children. The interface needs every node of the work tree, and the next Ready work unit, without knowing how projects store their children. WorkIterator only keeps the node it started on and the node it is on, and moves with getChild and getParent. Its readyOnly flag decides whether it stops on every node or only on Ready work units. TeamIterator gives a similar walk over the company: it collects the teams with a given profession and steps through them.

Chain of Responsibility with State. Who may work on a unit depends on its state. Each employee in the chain checks canAdvance, and the state decides which step is next and which level is needed.

9. Implementation

The interface works through ApplicationInterface. A user builds or loads a company and a workflow, picks a work unit and a team on the task page, and assigns the work. Each step lets the team's chain pick who does the work, moves the state on and tells the team.

Finding work. WorkIterator walks the project tree depth first. With readyOnly on, it only stops on work units in the Ready state. The facade uses it with readyOnly off to list every node and to register every node with the work mediator.

Finding a team. TeamIterator walks the departments and returns the teams whose profession matches. ApplicationInterface::getTeams uses it to list the teams that can do a unit.

Assigning. ApplicationInterface::checkAssignable refuses a team with the wrong profession, and moves a unit whose prerequisites are done from WaitingForPrerequisites to Ready. Team::assignWork keeps stepping the unit while it makes progress. If nobody in the team can move it on, Team::retryOrFail sends it back to Ready while it has retries left, and otherwise sets it to Failed. The facade also offers assignOnly and stepWork to move a unit on one state at a time, plus failWork, forceState and cancelWork.

Who does the work. handleWork calls canAdvance. Ready needs an intern, Ongoing needs a junior and Escalated needs a senior. compareCompetency checks the employee's level through the decorator chain. If the employee can't do it, the unit goes to the next member, in the order they were added to the team.

Doing the work. doWork runs the steps prepare, perform and finish. Each decorator adds its part of a step and passes it on to the employee it wraps, and the innermost employee tells the team when it is done. Then

WorkUnit::advanceState() lets the state object choose the next state, so each call moves the unit on by one state.

States. WorkNode holds a WorkState and passes advanceState to it. A work unit goes WaitingForPrerequisites, Ready, Ongoing, Escalated, Completed. When a unit completes, the work mediator calls update with its name on every other node. Each node removes that name from its prerequisites, and a node with none left moves to Ready. A node also keeps waiting while its parent project is waiting. When the last child of a project completes, Escalated sets the project to Escalated and advances it to Completed, unless the project has been cancelled.

Saving. ApplicationInterface::createMemento makes a Memento of the company or the workflow by cloning it, and setMemento restores one. FileSystem writes a memento to a plain text file and reads it back. The file uses tagged blocks: Department, Team, Member, Profession, Competency, Employee and Manager for the company, and Project, Unit and Prerequisites for the workflow. The interface saves them as company.txt and projects.txt.

Interface. A Qt 5 Widgets interface with a main menu, a company page (departments, teams, employees and their competency), a project page (projects, work units and prerequisites), a task page (assign, step, fail, force a state, cancel) and a load and save page. Every page calls ApplicationInterface.

History. The system does not keep a log of state changes. The task page shows the current state of every work unit, and each action reports the change it made.

10. Design decisions and revision history

Decisions 1 to 10 are from Practical 6. The rest were made while building the system. The full change logs are in diagrams/Design Decisions and Revision History/.

#	Date	Decision	Reason	Affected
1	Prac 6	Removed the workflow template	The tree of projects and work units is the workflow, so a separate template is not needed	Class diagram
2	Prac 6	WorkNode has 7 states	Tracks the lifecycle of work units	State diagram
3	Prac 6	Template Method has three steps	Every profession follows the same order and only fills in its own part	EmployeeComponent
4	Prac 6	WorkNode has a parent and manager operations	Completion has to move up the tree, and retries and escalation need something to act on	WorkNode, Manager
5	Prac 6	Observer also covers the parents	One mechanism handles unblocking the next unit and completing the parent	Class diagram

6	Prac 6	Dropped the abstract factory for the work tree	There is only one kind of tree, so a factory would add classes with nothing to vary	Class diagram
7	Prac 6	Dropped the decorator for access permission	Permission and who can do the work were the same question, so the chain of responsibility handles both	Class diagram
8	Prac 6	Dropped the company iterator	Printing the company is reporting, not execution	Class diagram
9	Prac 6	Phases are just Projects	A phase is a project node in the work tree, so it needs no class of its own	Project
10	Prac 6	The manager is a subclass of Employee	Escalation uses the existing successor chain, and Team stays a Mediator only	Manager
11	19 Oct	Members and parameters changed to pointers	Abstract classes can’t be held by value, so the generated code did not compile	All classes
12	22 Oct	Factory Method replaced by Builder and Director	An employee is made of several optional parts	EmployeeBuilder, ConcreteEmployeeBuilder, EmployeeDirector
13	24 Oct	Added clone() to the company and work trees	Mementos need full copies	WorkNode, CompanyNode, EmployeeComponent
14	25 Oct	Observer replaced by a work mediator	One mediator is simpler than a subject and observer on every node	WorkMediator, ConcreteWorkMediator
15	25 Oct	Professions stored by name instead of an id	WorkUnit and Team already use profession names	EmployeeProfession, EmployeeCompetency, WorkUnit
16	26 Oct	Team::addMember sets each member’s mediator and successor	Keeps the chain and mediator setup in one place	Team

17	29 Oct	Company iterator brought back as TeamIterator	Assigning work needs a team for a profession, which reverses decision 8	TeamIterator
18	29 Oct	One Memento class with a MementoType	The two memento classes were almost the same	Memento
19	30 Oct	Who works on a unit depends on its state: an intern can move Ready work, a junior Ongoing work and a senior Escalated work	Competency levels have to affect who does the work	EmployeeComponent::canAdvance
20	30 Oct	Failed work goes back to Ready until the retry limit	Escalated now means it needs a senior, not that it failed	Team::retryOrFail, Failed
21	31 Oct	Competency levels are a fixed enum: Intern, Junior and Senior	Only three levels are used, and an unknown level must be rejected	CompetencyLevel, EmployeeCompetency
22	31 Oct	Teams have a name	The interface lists, saves and reloads teams by name	Team
23	31 Oct	Members are compared by their innermost employee (getCore)	A member was told about its own work because the team compared the outer decorator with the inner employee	EmployeeComponent, Team
24	31 Oct	The facade registers every node with the work mediator (refreshWorkflow)	Nodes have to be colleagues of the mediator for prerequisites to be released	ApplicationInterface, WorkIterator

25	31 Oct	A node waits while its parent project is waiting	Work inside a project should not start before the project's own prerequisites are done	WorkNode
26	31 Oct	Team steps work one state at a time (stepWork, retryOrFail)	The interface shows each state change and retries work that nobody can move on	Team, ApplicationInterface
27	31 Oct	A Qt interface replaces the console program, built with qmake	Both industries need to be run from a usable interface	GUI, TIVIDY.pro
28	31 Oct	Both scenarios are built in the facade	One program runs either industry on the same engine	ApplicationInterface
29	1 Nov	Assigning checks the team's profession, and the scenarios build employees with the Builder	Wrong-team assignments were accepted silently, and the scenarios bypassed the Builder	ApplicationInterface
30	1 Nov	A project completes itself when the mediator reports that its last child is done	Escalated no longer completes the parent, so completion moves up the tree through the same mediator that releases prerequisites	Project, WorkNode

11. Build, run and test

You need g++ with C++11, Qt 5 (qtbase5-dev and qt5-qmake) and make. The tests use Catch2 (src/catch.hpp), which is already in the repository.

Clone Project:

git clone https://github.com/Brett-Pitts/COS214Project

Go to the project directory

cd COS214Project

Build Docker Image

docker build -t tividy .

Run Docker

```
docker run -it \
  -e DISPLAY=$DISPLAY \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  tividy /bin/bash
```

Compile

qmake GUI/TIVIDY.pro

make -j$(nproc)

Run

./GUI/TIVIDY

Run ScenarioA

./GUI/TIVIDY ScenarioA

Run ScenarioB

./GUI/TIVIDY ScenarioB

Run Tests

make -C src test

make -C src clean

12. Engineering evidence

Github:



Note: Jamie's and Brett's code additions and deletions are very high from working on GUI, doxygen and moving files around multiple times.

Documentation. Doxygen comments on the classes and important methods. The generated documentation is in docs/ (docs/index.html).

Bonus work. You can save a company or workflow structure to a file to a file to be loaded later. Auto saves and loads structures upon closing and reopening app.

13. Contribution statement

Rudolph Botha: Implemented the Decorator for employees (professions and competency levels) and the Iterator for the work tree. Drew the sequence diagrams and the state diagram, and updated Activity Diagram 2 to match the current state of the program. Fixed bugs in the work mediator, the facade and the competency levels. Wrote tests for the decorators, company structure and iterator.

Francois Venter: Worked on Company composite, team mediator, worknode mediator. Made the final activity diagram 1.Made tests for Company composite. Did the research and created the industry scenario. General bugfixing.

Louwrens Johansen: Built the chain of responsibility, Worknode composite, fixed header types and inheritance logic. Changed director to match Design pattern logic. Made the initial class diagram and updated it to match the current state of the program. Wrote tests for chain logic.

Brett Pitts: Wrote the State implementations and WorkMediator, and wrote tests for both. Created initial Activity Diagrams 1-3, the GUI mockup, and the PowerPoint. Then got the GUI working. Added extensive Doxygen documentation throughout.Also created readme.

Jamie King: Created the main design for the system, delegated work to team members. Implemented the Builder and Memento design pattern. Implemented the system that allows one to save either a workflow or company composite structure to a file to allow loading into a previous state later. Added tests to assert creating memento(also cloning composites) creates a structure identical to the original